2. Computer systems performance

2.2. Performance metrics

Execution time → system execution time (disk accesses, mem. accesses, I/O activities, OS overhead) => (system performance)

→ CPU execution time => CPU performance

CPU time — workloads (mixture of programs)

=> predict system performance on a given program

2.3 Benchmarks

- include real applications (e.g., compiler, spreadsheet app, etc.)
  - → kernels
  - → toy programs
  - → synthetic benchmarks
- combinations / mixes of real applications => benchmark suites

$$A > B \ \& \ B > C \Rightarrow A > C$$

- SPEC – Standard Performance Evaluation Corporation

www.spec.org

1989, 92, 95, 2000, 2006, 2017

2.3.1 Desktop benchmarks

- CPU SPEC => SPEC CPU Suite
- Graphics
- Java

SPEC

SPEC2006 benchmark description	Benchmark name by SPEC generation				
	SPEC2006	SPEC2000	SPEC95	SPEC92	SPEC89
GNU C compiler					gcc
Interpreted string processing			perl		espresso
Combinatorial optimization		mcf			li
Block-sorting compression		bzip2		compress	eqntott
Go game (AI)	go	vortex	go	sc	
Video compression	h264avc	gzip	ijpeg		
Games/path finding	astar	eon	m88ksim		
Search gene sequence	hmmer	twolf			
Quantum computer simulation	libquantum	vortex			
Discrete event simulation library	omnetpp	vpr			
Chess game (AI)	sjeng	crafty			
XML parsing	xalancbmk	parser			
CFD/blast waves	bwaves				fpppp
Numerical relativity	cactusADM				tomcatv
Finite element code	calculix				doduc
Differential equation solver framework	dealII				nasa7
Quantum chemistry	gamess				spice
EM solver (freq/time domain)	GemsFDTD			swim	matrix300
Scalable molecular dynamics (~NAMD)	gromacs		apsi	hydro2d	
Lattice Boltzman method (fluid/air flow)	lbm		mgrid	su2cor	
Large eddie simulation/turbulent CFD	LESlie3d	wupwise	applu	wave5	
Lattice quantum chromodynamics	milc	apply	turb3d		
Molecular dynamics	namd	galgel			
Image ray tracing	povray	mesa			
Spare linear algebra	soplex	art			
Speech recognition	sphinx3	equake			
Quantum chemistry/object oriented	tonto	facerec			
Weather research and forecasting	wrf	ammp			
Magneto hydrodynamics (astrophysics)	zeusmp	lucas			
		fma3d			
		sixtrack			

2.3.2 Server benchmarks

- Throughput-centered performance
- Convert CPU time into a rate
- ~~mix~~ CPU SPEC suites
- Transaction Processing Council www.tpc.org

TPC benchmarks

2.3.3 Reporting performance results

- Fundamental principle — reproducibility

2.3.4 Summarizing performance results

SPEC Ratio

Example: SPEC Ratio on Computer A was 1.25 higher than Computer B

$$\frac{\text{SPECRatio}_A}{\text{SPECRatio}_B} = \frac{\dfrac{\cancel{\text{Execution time reference}}}{\text{Execution time}_A}}{\dfrac{\cancel{\text{Execution time reference}}}{\text{Execution time}_B}} = \frac{\text{Execution time}_B}{\text{Execution time}_A} =$$

$$= \frac{\text{Performance}_A}{\text{Performance}_B}$$

$$\text{Geometric mean} = \sqrt[n]{\prod_{i=1}^{n} \text{Sample}_i}$$

$\text{Sample}_i \rightarrow \text{SPECRatio}_i$

① The geometric mean of ratios = the ratio of geometric means

② The ratio of geometric means = the geometric mean of the performance ratios

$$\frac{\text{Geometric mean}_A}{\text{Geometric mean}_B} = \frac{\sqrt[n]{\prod_{i=1}^{n} \text{SPECRatio}_{Ai}}}{\sqrt[n]{\prod_{i=1}^{n} \text{SPECRatio}_{Bi}}} =$$

$$= \sqrt[n]{\prod_{i=1}^{n} \frac{\text{SPECRatio}_{Ai}}{\text{SPECRatio}_{Bi}}} = \sqrt[n]{\prod_{i=1}^{n} \frac{\dfrac{\cancel{\text{Execution time reference}_i}}{\text{Execution time}_{Ai}}}{\dfrac{\cancel{\text{Execution time reference}_i}}{\text{Execution time}_{Bi}}}} =$$

$$= \sqrt[n]{\prod_{i=1}^{n} \frac{\text{Execution time}_{Bi}}{\text{Execution time}_{Ai}}} = \sqrt[n]{\prod_{i=1}^{n} \frac{\text{Performance}_{Ai}}{\text{Performance}_{Bi}}}$$

Extremistan

<u>Mediocristan</u> (performance) → $\text{stdev} = \sqrt{\sum_{i=1}^{n} \left(\left(\text{Sample}_i\right) - \text{Mean}\right)^2}$

SPEC Ratio$_i$

<u>geometric standard deviation</u>

$$\text{Geometric mean} = \exp\left(\frac{1}{n} \times \sum_{i=1}^{n} \ln(\text{Sample}_i)\right)$$

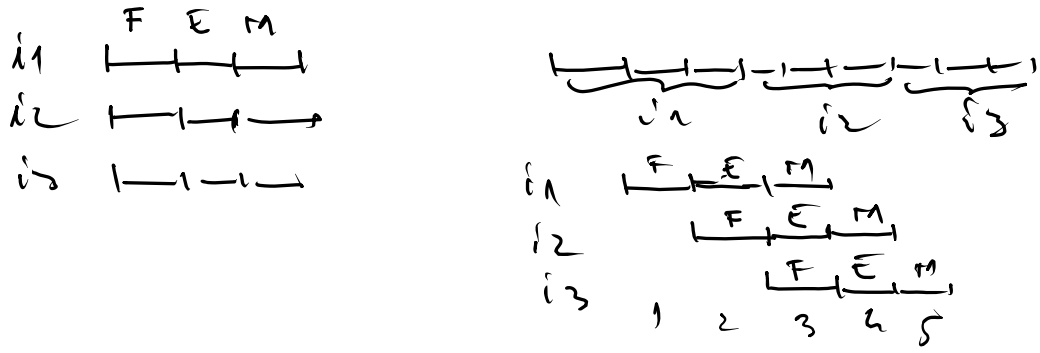
$$\text{gstdev} = \exp\left(\sqrt{\frac{1}{n} \cdot \sum_{i=1}^{n} \left[\ln(\text{Sample}_i) - \ln(\text{Geometric mean})\right]^2}\right)$$

$\boxed{2.4}$ Quantitative Principles of Computer Design

- SLD – System-Level Design

① <u>Take advantage of parallelism</u>

- <u>Instruction-Level Parallelism</u> → Pipelining instructions



- Carry-Lookahead Adders
- Multi-core system

② Principle of locality

Obs Programs tend to reuse data and instructions they have used recently

Rule of thumb

A program spends 90% of of its execution time on 10% of the code

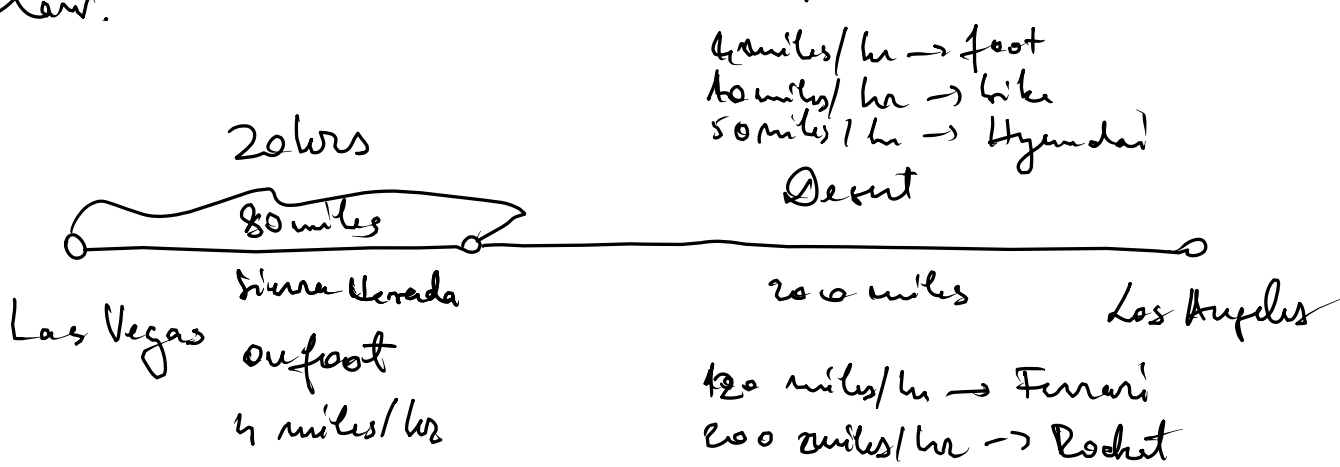
- Mostly applies to instructions
- But also works on data

2 types of locality observed → temporal locality / spatial locality

③ Focus on the common case

Amdahl's law

The performance gain that we can obtain by improving one part of the system can be computed with Amdahl's law.



car2

Vehicle for 2nd part	Hrs for 2nd part	Speedup in Desert	Hrs for entire trip	Speedup entire trip
Feet	50	1	70	1
Bike	20	2.5	40	1.8
Hyundai	4	12.5	24	2.9
Ferrari	1.67	30	21.67	3.2
Rocket car	1	50	21	3.3

$$\text{Speedup entire syst.} = \frac{\text{Performance of task using the speedup when possible}}{\text{Performance of task without the speedup}} =$$

$$= \frac{\text{Exec. time of task without speedup}}{\text{Exec time of task using speedup when possible}} =$$

$$= \frac{\text{Exec time old}}{\text{Execution time old}\left[(1-\text{Fraction enhanced}) + \frac{\text{Fraction enhanced}}{\text{Speedup enhanced}}\right]} =$$

$$= \frac{1}{\underbrace{(1-\text{Fraction enhanced})} + \underbrace{\frac{\text{Fraction enhanced}}{\text{Speedup enhanced}}}}$$

$\frac{200}{280}$

Speedup enhanced → 50 for Rocket car

The limit of system enhancement

$$\frac{1}{1 - \text{Fraction enhanced}}$$

2.5 Computer Performance Equation